

# 1 Отчет по задаче 2: Описание работы системы DiGr DSL

## 1.1 Назначение системы

DiGr — инструмент для поиска фрагментов текстового документа с помощью декларативного языка запросов. Документ предварительно разбирается в иерархическое дерево (AST), над которым затем выполняются запросы трёх типов: выборка узлов (FIND), поиск контекстных окон (CONTEXT) и измерение расстояний между парами узлов (DISTANCE).

Общий поток обработки:

исходный файл → AstDocument → текст запроса → DslQuery (AST) → результат (JSON)

## 1.2 Структура исходного кода

Исходный код расположен в директории `src/` и разделён на три независимые подсистемы с однонаправленными зависимостями:

| Пакет                          | Назначение                                                                                                        |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>src/actor/</code>        | Базовый actor runtime: FSM-акторы, Mailbox, планировщики. Не знает ни про документы, ни про DSL.                  |
| <code>src/document_ast/</code> | Построение AstDocument из исходного файла по YAML-конфигурации формата. Использует <code>actor/</code> .          |
| <code>src/dsl/</code>          | Разбор DSL-запроса и его исполнение по AstDocument. Использует <code>actor/</code> и <code>document_ast/</code> . |

## 1.3 Конфигурация форматов документов

Структура конкретного формата документа задаётся YAML-файлом в `config/formats/`. Для текстового формата используется `txt.yaml`, который определяет иерархию сущностей:

document → page → paragraph → sentence → clause → word

Каждый уровень описывает: имя сущности, правило сегментации текста (`passthrough`, `split` или `match`) и отношение вложения `contains`. Загрузку и валидацию конфигурации выполняет `src/document_ast/config/config_loader.py`, проверяющий в том числе наличие циклов в иерархии сущностей. Имена сущностей в DSL-запросах не захардкожены: они берутся из конфигурации конкретного формата. Допустимость имени проверяется на этапе исполнения запроса.

## 1.4 Подсистема DSL

Подсистема реализована в `src/dsl/` и состоит из шести слоёв.

| Директория        | Содержимое                                                                                                         |
|-------------------|--------------------------------------------------------------------------------------------------------------------|
| api/              | Публичные точки входа: <code>ActorDslEngine</code> , <code>ActorDslParser</code> , <code>ActorDslExecutor</code> . |
| model/            | Типизированный AST запроса ( <code>query_ast.py</code> ).                                                          |
| parsing/          | Лексер и синтаксический анализатор.                                                                                |
| execution/        | Индексация AST, вычисление предикатов, расстояния, валидация.                                                      |
| actors/parsing/   | Актеры конвейера разбора запроса.                                                                                  |
| actors/execution/ | Актеры конвейера исполнения запроса.                                                                               |

#### 1.4.1 Публичный API

Внешний код работает с тремя классами из `src/dsl/api/`:

- `ActorDslParser` (`query_parser.py`) — принимает строку запроса, возвращает `DslQuery`;
- `ActorDslExecutor` (`query_executor.py`) — принимает `DslQuery` и `AstDocument`, возвращает результат;
- `ActorDslEngine` (`engine.py`) — объединяет оба шага, принимает строку запроса и документ напрямую.

`ActorDslEngine.execute()` реализован в три строки: вызывает парсер, затем исполнитель. Это точка входа для большинства сценариев использования.

#### 1.4.2 Модель запроса

Результат разбора хранится в типизированных датаклассах из `src/dsl/model/query_ast.py`. Верхний тип:

```
DslQuery = ContextQuery | FindQuery | DistanceQuery
```

Выражения в `WHERE` и предикатах селекторов представлены типом:

```
Expression = ComparisonExpression | FunctionExpression | NotExpression |
              BinaryExpression
```

Вспомогательные узлы: `Selector`, `Pattern`, `SpanSpec`, `WithinConstraint`, `FieldRef`, `CountConstraint`, `PairLimit`, `DistanceReturn`. Все классы реализуют метод `to_dict()` через базовый класс `Serializable`.

#### 1.4.3 Лексический анализ

Лексер реализован в классе `DslLexer` (`src/dsl/parsing/lexer.py`). Метод `tokenize()` обходит строку посимвольно и делегирует чтение отдельным методам:

При лексической ошибке выбрасывается `DslSyntaxError` (`parsing/errors.py`) с полями `offset`, `line`, `column`.

#### 1.4.4 Синтаксический анализ

Парсер реализован в классе `DslTokenStreamParser` (`src/dsl/parsing/recursive_descent_parser.py`) методом рекурсивного спуска. Точка входа — `parse_query()` (стр. 76), которая ветвится по первому токену потока.

| Метод                           | Стр. | Что распознаёт                                                                                                                                                           |
|---------------------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>_read_identifier()</code> | 113  | Идентификаторы и ключевые слова. После сборки лексемы выполняет <code>lexeme.upper()</code> и ищет в словаре <code>_KEYWORDS</code> — ключевые слова регистронезависимы. |
| <code>_read_integer()</code>    | 128  | Целые неотрицательные числа.                                                                                                                                             |
| <code>_read_string()</code>     | 138  | Строковые литералы в двойных кавычках. Escape-последовательности обрабатывает <code>_read_string_escape()</code> (стр. 159).                                             |
| <code>_read_regex()</code>      | 181  | Регулярные выражения в слешах. Допустимые флаги: <code>i</code> , <code>m</code> , <code>s</code> , <code>u</code> .                                                     |
| <code>_match_operator()</code>  | 220  | Операторы. Перебирает словарь <code>_OPERATORS</code> в порядке убывания длины, чтобы <code>!~</code> распознавалось раньше <code>!=</code> .                            |

### 1.4.5 Слой исполнения

Исполнение запроса реализовано в `src/dsl/execution/`. Перед запуском запроса строится `DocumentIndex`, который используется всеми компонентами исполнения.

**DocumentIndex** (`document_index.py`). При инициализации обходит дерево `AstDocument` и строит:

- список всех узлов, отсортированный по `(start, end, entity)`;
- словарь `_nodes_by_entity`: имя сущности → список узлов;
- словарь `_parent_by_id`: `id(node)` → родительский узел.

Основные методы: `nodes_of_entity()`, `children()`, `descendants()`, `ancestors()`, `nodes_within_span()`, `container_nodes_for_span()`, `previous_node()`, `next_node()`.

**PredicateEvaluator** (`predicate_evaluator.py`). Вычисляет логические выражения из `WHERE` и предикатов селекторов. Метод `evaluate(node, expression)` рекурсивно обходит AST выражения. Реализованные функции:

Операторы сравнения: `=`, `!=`, `<`, `<=`, `>`, `>=`, `~=` (regex match), `!~=` (regex not match).  
Доступные поля узла: `entity`, `text`, `start`, `end`, `metadata.<field>`.

**DistanceCalculator** (`distance_calculator.py`). Вычисляет расстояние между двумя узлами в единицах указанной сущности. Использует `bisect_left` по отсортированному списку узлов для подсчёта промежуточных элементов. Также управляет отбором пар согласно режиму `LIMIT_PAIRS`: `nearest`, `all_nearest`, `all` или `k` ближайших.

**QueryValidator** (`query_validator.py`). Проверяет, что все имена сущностей из запроса присутствуют в `DocumentIndex`. Выполняется до запуска исполнения; при ошибке возвращает понятное сообщение, а не неопределённое поведение на этапе вычисления предикатов.

### 1.4.6 Акторный конвейер

Разбор и исполнение запроса организованы по одному шаблону: координатор раздаёт задачи воркерам, воркеры отправляют результаты коллектору.

| Правило             | Метод / стр.                                  | Примечания                                                                                                                                                       |
|---------------------|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <Запрос>            | <code>parse_query()</code> / 76               | Ветвление по CONTEXT / FIND / DISTANCE. Точка с запятой необязательна.                                                                                           |
| <Запрос CONTEXT>    | <code>parse_context_query()</code> / 99       | Порядок: FOR, WITHIN*, WHERE?, RETURN?.                                                                                                                          |
| <Запрос FIND>       | <code>parse_find_query()</code> / 107         | Порядок: WHERE?, WITHIN*, RETURN? — порядок WHERE и WITHIN обратный по сравнению с CONTEXT.                                                                      |
| <Запрос DISTANCE>   | <code>parse_distance_query()</code> / 114     | Два селектора через токен TO, затем WITHIN*, LIMIT_PAIRS?.                                                                                                       |
| <Спецификация окна> | <code>parse_span_spec()</code> / 215          | <code>entity[ор число]</code> : квадратные скобки содержат числовое ограничение, не логическое выражение.                                                        |
| <Блок LIMIT_PAIRS>  | <code>parse_limit_pairs_clause()</code> / 164 | При отсутствии возвращает <code>PairLimit(mode="nearest")</code> по умолчанию.                                                                                   |
| <Блок RETURN>       | <code>parse_return_clause()</code> / 177      | Принимает любой идентификатор; <code>distance(entity)</code> распознаётся отдельно в <code>parse_return_item()</code> (стр. 185).                                |
| <Отрицание>         | <code>parse_negation()</code> / 248           | Рекурсивный вызов: допускает NOT NOT expr.                                                                                                                       |
| <Предикат>          | <code>parse_predicate()</code> / 253          | Функция отличается от сравнения двухтокенным lookahead: IDENTIFIER + LPAREN.                                                                                     |
| <Аргумент>          | <code>parse_argument()</code> / 283           | Спецификация окна и селектор различаются через <code>_looks_like_span_spec()</code> (стр. 342): если за [ идёт оператор количества — это <code>SpanSpec</code> . |

Конвейер разбора (actors/parsing/).

Конвейер исполнения (actors/execution/).

## 1.5 Типы запросов

### 1.5.1 FIND

Выбирает все узлы указанной сущности, удовлетворяющие условию WHERE.

```

FIND sentence
WHERE has_descendant(word[text ~= /телефон/i])
RETURN text, count

```

| Функция                               | Семантика                                                                                                 |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>has_child(selector)</code>      | Хотя бы один прямой потомок узла соответствует <code>selector</code> .                                    |
| <code>has_descendant(selector)</code> | Хотя бы один потомок на любой глубине соответствует <code>selector</code> .                               |
| <code>has_ancestor(selector)</code>   | Хотя бы один предок соответствует <code>selector</code> .                                                 |
| <code>contains(arg)</code>            | Текст узла содержит строку, соответствует <code>regex</code> или содержит узел по <code>selector</code> . |
| <code>follows(selector)</code>        | Предыдущий соседний узел того же типа соответствует <code>selector</code> .                               |
| <code>precedes(selector)</code>       | Следующий соседний узел того же типа соответствует <code>selector</code> .                                |
| <code>intersects(selector)</code>     | Span узла пересекается со span узла из <code>selector</code> .                                            |
| <code>inside(selector)</code>         | Текущий узел находится внутри контейнера из <code>selector</code> .                                       |

| Актор                                     | Роль                                                                                                                                                                                              |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>DslParserCoordinatorActor</code>    | Управляет последовательностью: запускает лексер, затем парсер. FSM-состояния: <code>IDLE</code> , <code>WAITING_FOR_TOKENS</code> , <code>WAITING_FOR_QUERY_AST</code> , <code>COMPLETED</code> . |
| <code>DslLexerActor</code>                | Вызывает <code>DslLexer.tokenize()</code> , отправляет токены координатору.                                                                                                                       |
| <code>DslQueryParserActor</code>          | Вызывает <code>DslTokenStreamParser.parse_query()</code> , строит AST. Фазовые состояния отражают стадию разбора.                                                                                 |
| <code>DslParseResultCollectorActor</code> | Сохраняет результат или ошибку.                                                                                                                                                                   |

| Актор                                         | Роль                                                                                                                                                                                                                   |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>DslExecutionCoordinatorActor</code>     | Валидирует запрос, раздаёт задачи воркерам, собирает частичные результаты. Состояния: <code>IDLE</code> , <code>EVALUATING_FIND_CANDIDATES</code> , <code>EVALUATING_CONTEXT_WINDOWS</code> , <code>COMPLETED</code> . |
| <code>DslExecutionWorkerActor</code>          | Вычисляет соответствие для одного кандидата ( <code>FIND</code> ) или одного стартового узла ( <code>CONTEXT</code> ).                                                                                                 |
| <code>DslExecutionResultCollectorActor</code> | Сохраняет финальный результат или ошибку.                                                                                                                                                                              |

Порядок исполнения:

1. Из индекса берутся все узлы сущности (`nodes_of_entity()`).
2. Каждый узел проверяется предикатом `WHERE` через `PredicateEvaluator.evaluate()`.
3. Прошедшие фильтр узлы формируют результат `FindQueryExecutionResult`.
4. `RETURN` определяет поля в выходном JSON, но не меняет состав совпадений.

`WITHIN` в `FIND` дополнительно ограничивает результат по структурным контейнерам: узел сохраняется только если его охватывает ровно столько контейнеров указанного типа, сколько задано ограничением.

### 1.5.2 CONTEXT

Ищет минимальные непрерывные окна из соседних узлов базовой сущности, внутри которых встречаются все перечисленные паттерны.

```
CONTEXT sentence [<=4]
FOR "дверь", "тишина"
WITHIN paragraph [=1]
RETURN text, matches
```

Контекстное окно — это подряд идущие узлы одного типа без пропусков. Для базовой сущности из  $n$  узлов  $S_1, \dots, S_n$  допустимы все окна  $S_i, S_i..S_{i+1}, \dots, S_i..S_{i+k}$ , удовлетворяющие ограничению длины.

Порядок исполнения:

1. Берутся все узлы базовой сущности.
2. Перебираются окна допустимой длины.
3. Окно проверяется на все паттерны из FOR: строковый литерал — поиск подстроки в тексте окна; regex — поиск совпадения; селектор — наличие узла с предикатом внутри окна.
4. Проверяются ограничения WITHIN.
5. Если задан WHERE — вычисляется для найденного окна.
6. Из набора прошедших проверку окон оставляются только минимальные: если окно  $A$  содержит окно  $B$  с теми же совпадениями,  $A$  отбрасывается.

Паттерны в FOR могут иметь алиасы (a: word[text = "ключ"]), которые сохраняются в поле matches результата.

### 1.5.3 DISTANCE

Измеряет расстояние между парами узлов в единицах указанной сущности.

```
DISTANCE definition[metadata.index ~= "зык"]
TO definition[metadata.index ~= "зык"]
LIMIT_PAIRS all
RETURN pairs, stats, distance(definition), count
```

Аргумент distance(entity) задаёт единицу измерения: distance(symbol) считает символы, distance(sentence) — предложения между элементами пары. Расстояние — количество узлов указанной сущности строго между левым и правым элементами пары.

LIMIT\_PAIRS управляет объёмом результата:

| Значение              | Поведение                           |
|-----------------------|-------------------------------------|
| отсутствует / nearest | Одна ближайшая пара.                |
| all_nearest           | Все пары с минимальным расстоянием. |
| all                   | Все допустимые пары.                |
| целое число $k$       | $k$ ближайших пар.                  |

Результат содержит поля pairs, stats (count, mean, variance, min, max), count. Статистика вычисляется по всем парам до применения LIMIT\_PAIRS.